Universidad Autónoma de Tamaulipas
Unidad Académica Multidisciplinaria Mante Centro

Nombre del trabajo:
Reporte De Practica
LAB01

Asignatura:
Diseño Electrónico Basado En Sistemas Embebidos

Profesor:
Ing. López Piña Daniel

Alumnos:
Carreon Hernández Emmanuel
Castillo Hernández Rebeca
Medellin López Martin
Rodríguez Osorio Diego
Salzar Lares Aylin Ximena
Torres Juarez Humberto

Grado: 8° **Grupo:** J

PRÁCTICA 1. Hola Mundo Embebido — Blink y Monitor Serial

Introducción

En esta práctica se trabajó con uno de los ejercicios más básicos, pero también más importantes dentro del desarrollo de sistemas embebidos: el encendido y apagado controlado de un LED mediante un microcontrolador Arduino. Aunque a simple vista parece una actividad sencilla, en realidad permite comprender conceptos fundamentales que serán utilizados en proyectos más complejos, como la configuración de pines de entrada y salida, la estructura de un programa en Arduino y el manejo del tiempo de ejecución.

Además del control del LED, se utilizó el Monitor Serial para observar mensajes generados por el programa en tiempo real. Esto permitió comprobar el funcionamiento del código y entender que un sistema embebido no solo ejecuta acciones físicas, sino que también puede comunicar información útil al usuario o al desarrollador. Con ello, la práctica sirvió como una primera aproximación al comportamiento de un sistema programable y a la forma en que interactúa con el entorno.

VERDAD, BELLEZA, PROBIDAD

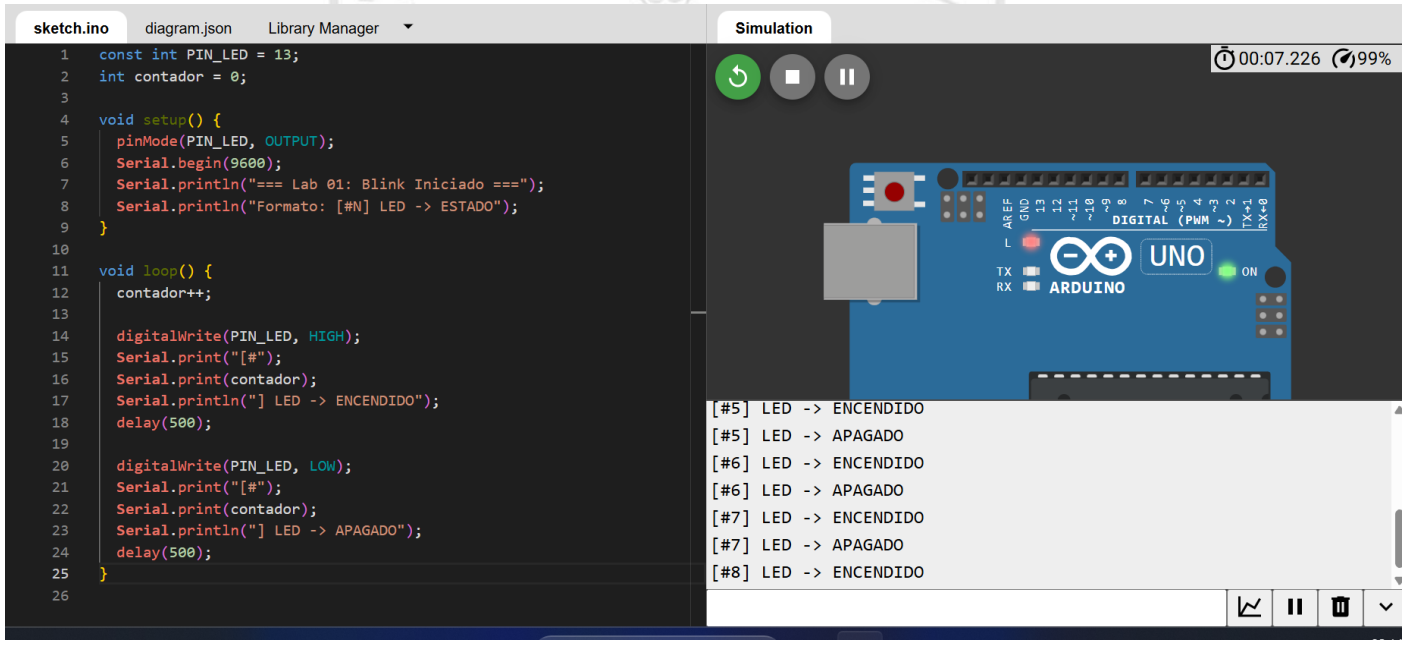
Desarrollo

Parte base — Blink y Monitor Serial

La parte base inició con la creación de un nuevo proyecto en Wokwi utilizando una tarjeta Arduino Uno. Para simplificar el montaje y enfocarse en la lógica del programa, se aprovechó el LED integrado del pin 13, recurso incluido directamente en la placa. Una vez abierto el entorno, se escribió el código correspondiente al laboratorio, donde se estableció una constante para el pin del LED y se declaró una variable entera que funcionó como contador de parpadeos.

En la función setup() se configuró el pin como salida y se activó la comunicación serial a una velocidad de 9600 baudios. Esto permitió que, además del comportamiento visual del LED, el sistema proporcionara información textual de cada cambio de estado. Después, en loop(), se programó una secuencia cíclica en la que el LED se encendía con digitalWrite(HIGH), esperaba 500 milisegundos, se apagaba con digitalWrite(LOW) y esperaba otros 500 milisegundos antes de repetir el ciclo.

Cada vez que el LED cambiaba de estado, se enviaba un mensaje al Monitor Serial indicando el número de ciclo y si el LED estaba encendido o apagado. De este modo, el usuario podía observar simultáneamente el resultado físico de la práctica y la salida de depuración en pantalla. Al poner en marcha la simulación, el LED comenzó a parpadear de forma constante y el Monitor Serial mostró la información esperada, lo cual confirmó el funcionamiento correcto del programa y permitió entender la relación entre la estructura del código y la respuesta del sistema.



Parte 1 — Cambio de tiempos de encendido y apagado

Desarrollo

En esta parte se tomó como base el mismo programa inicial de parpadeo, pero se modificaron los valores de temporización con el objetivo de analizar cómo cambia la apariencia visual del LED cuando el tiempo de activación y desactivación deja de ser simétrico. Para ello, se conservaron las mismas instrucciones de encendido y apagado, así como la impresión de mensajes en el Monitor Serial, pero se sustituyeron los retardos de 500 milisegundos por un encendido de 200 milisegundos y un apagado de 800 milisegundos.

Al ejecutar esta versión del programa, se observó claramente que el LED emitía destellos mucho más breves y permanecía apagado durante más tiempo. Esto hizo que el parpadeo dejara de percibirse uniforme y se volviera desigual. El ejercicio permitió comprobar que la señal luminosa no solo depende de la lógica de encender y apagar, sino de cuánto tiempo permanece en cada estado. Además, al mantener el uso del Monitor Serial, fue posible seguir observando el incremento del contador y el cambio de estado del LED, verificando que el programa seguía funcionando correctamente pese a la modificación.

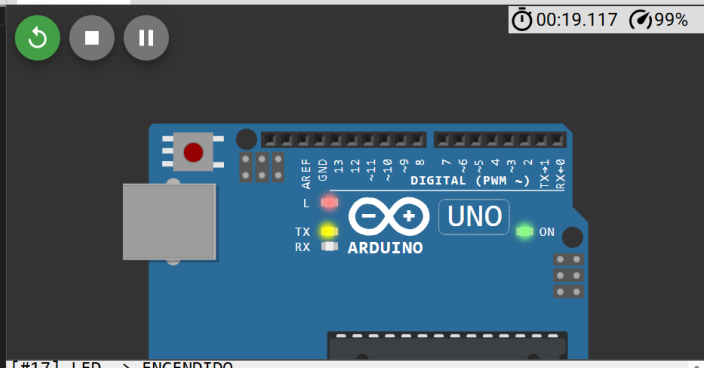
Este desarrollo reforzó el concepto de temporización en sistemas embebidos, mostrando cómo pequeños cambios en los valores programados se reflejan inmediatamente en la salida física del sistema.

sketch.ino | diagram.json | Library Manager

```
const int PIN_LED = 13;
int contador = 0;

void setup() {
  pinMode(PIN_LED, OUTPUT);
  Serial.begin(9600);
  Serial.println("=== Ejercicio 1 ===");
  Serial.println("LED 200 ms encendido / 800 ms apagado");
}

void loop() {
  contador++;

  digitalWrite(PIN_LED, HIGH);
  Serial.print("[#");
  Serial.print(contador);
  Serial.println("] LED -> ENCENDIDO");
  delay(200);

  digitalWrite(PIN_LED, LOW);
  Serial.print("[#");
  Serial.print(contador);
  Serial.println("] LED -> APAGADO");
  delay(800);
}
```

Simulation

00:19.117 99%




```
[#17] LED -> ENCENDIDO
[#17] LED -> APAGADO
[#18] LED -> ENCENDIDO
[#18] LED -> APAGADO
[#19] LED -> ENCENDIDO
[#19] LED -> APAGADO
[#20] LED -> ENCENDIDO
```

VERDAD, BELLEZA, PROBIDAD

Parte 2 — Patrón SOS

Desarrollo

Para esta parte se diseñó una secuencia más compleja que la del simple parpadeo, con el propósito de representar el mensaje SOS del código Morse utilizando únicamente el encendido y apagado del LED. Se partió de la idea de que un pulso corto y uno largo pueden diferenciarse mediante la duración del encendido, por lo que se implementaron intervalos de 200 milisegundos para los destellos cortos y de 600 milisegundos para los destellos largos. Entre cada destello se dejó una pausa breve, y al final de la secuencia completa se agregó una espera más prolongada de dos segundos antes de reiniciar.

En la programación se organizaron las llamadas correspondientes para formar la secuencia de tres pulsos cortos, tres pulsos largos y nuevamente tres pulsos cortos. Una vez ejecutado el código en Wokwi, se observó que el LED reproducía de forma visible el patrón temporal establecido. La diferencia entre pulsos cortos y largos fue apreciable, y la pausa entre secuencias ayudó a distinguir cada repetición del mensaje. También se incorporó un mensaje en el Monitor Serial indicando que el patrón SOS había sido enviado, lo que permitió documentar el comportamiento de la simulación.

Este ejercicio mostró que un sistema embebido no solo puede realizar tareas repetitivas, sino también producir secuencias organizadas que transmiten información. Gracias a ello, se reforzó la comprensión del control temporal como una herramienta útil para codificar señales o eventos.

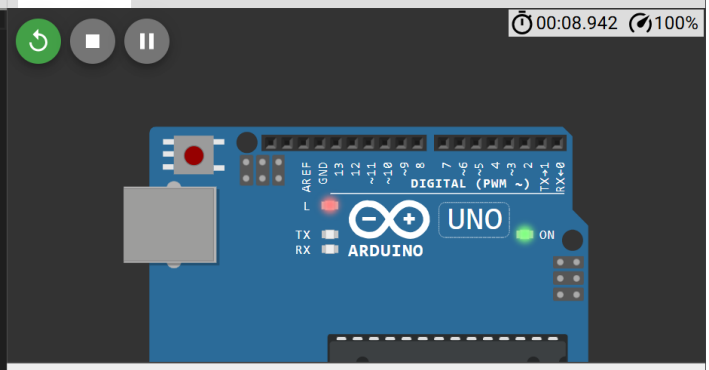
WOKWI SAVE SHARE LAB01_Parte2 Docs

sketch.ino | diagram.json | Library Manager

```
const int PIN_LED = 13;

void corto() {
  digitalWrite(PIN_LED, HIGH);
  delay(200);
  digitalWrite(PIN_LED, LOW);
  delay(200);
}

void largo() {
  digitalWrite(PIN_LED, HIGH);
  delay(600);
  digitalWrite(PIN_LED, LOW);
  delay(200);
}

void setup() {
  pinMode(PIN_LED, OUTPUT);
  Serial.begin(9600);
  Serial.println("=== Ejercicio 2 ===");
  Serial.println("Patron SOS iniciado");
}

void loop() {
  corto();
  corto();
  corto();
```

Simulation

00:08.942 100%




```
=== Ejercicio 2 ===
Patron SOS iniciado
SOS enviado
```

VERDAD, BELLEZA, PROBIDAD

Parte 3 — Uso de millis()

Desarrollo

En esta parte se modificó el programa base de parpadeo para incluir el valor devuelto por la función millis(), cuya utilidad consiste en indicar el tiempo transcurrido, en milisegundos, desde que el microcontrolador comenzó la ejecución del sketch. Para ello, antes de imprimir el estado del LED en el Monitor Serial, se incorporó la lectura de millis() dentro de la cadena de salida, de forma que cada mensaje indicara también el instante en que el LED se encendía o apagaba.

Al ejecutar la simulación, el Monitor Serial comenzó a mostrar líneas con el número de ciclo, el estado del LED y el valor del tiempo acumulado. Esto permitió observar que los valores aumentaban progresivamente y guardaban relación con la secuencia de encendido y apagado programada. Por ejemplo, si el LED se encendía y apagaba cada 500 milisegundos, los tiempos mostrados coincidían aproximadamente con dichos intervalos.

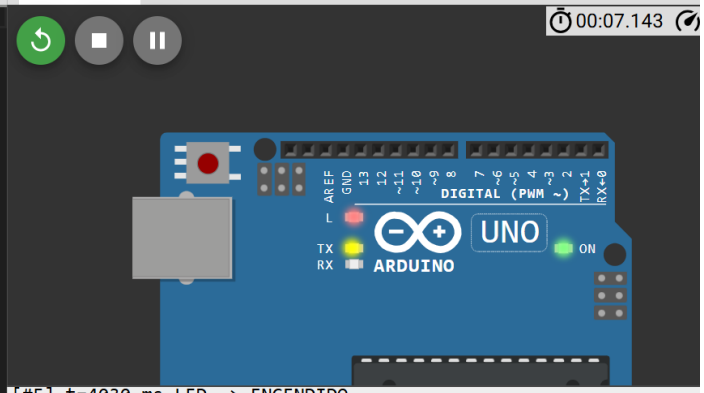
La incorporación de millis() permitió visualizar que el sistema lleva un control interno del tiempo independiente de las salidas físicas, y que este recurso puede ser aprovechado para sincronizar eventos, medir intervalos o desarrollar programas más avanzados. Así, el ejercicio amplió la comprensión del manejo temporal en sistemas embebidos y sentó una base importante para prácticas posteriores.

WOKWI SAVE SHARE LAB01_Parte3 Docs

sketch.ino | diagram.json | Library Manager

```
const int PIN_LED = 13;
int contador = 0;

void setup() {
  pinMode(PIN_LED, OUTPUT);
  Serial.begin(9600);
  Serial.println("=== Ejercicio 3 ===");
  Serial.println("Formato: [#N] t=XXXX ms LED -> ESTADO");
}

void loop() {
  contador++;

  digitalWrite(PIN_LED, HIGH);
  Serial.print("[#");
  Serial.print(contador);
  Serial.print("] t=");
  Serial.print(millis());
  Serial.println(" ms LED -> ENCENDIDO");
  delay(500);

  digitalWrite(PIN_LED, LOW);
  Serial.print("[#");
  Serial.print(contador);
  Serial.print("] t=");
  Serial.print(millis());
  Serial.println(" ms LED -> APAGADO");
```

Simulation 00:07.143




```
[#5] t=4030 ms LED -> ENCENDIDO
[#5] t=4530 ms LED -> APAGADO
[#6] t=5030 ms LED -> ENCENDIDO
[#6] t=5531 ms LED -> APAGADO
[#7] t=6031 ms LED -> ENCENDIDO
[#7] t=6532 ms LED -> APAGADO
[#8] t=7032 ms LED -> ENCENDIDO
```

VERDAD, BELLEZA, PROBIDAD

Conclusión

En esta práctica se comprendieron los fundamentos más importantes del trabajo con Arduino y sistemas embebidos, ya que se aprendió a estructurar un programa con setup() y loop(), controlar una salida digital y utilizar el Monitor Serial como medio de seguimiento del comportamiento del sistema. A través del parpadeo del LED se confirmó la relación directa entre las instrucciones programadas y la respuesta física observada en la simulación.

Además, las actividades de modificación permitieron ampliar este aprendizaje al experimentar con diferentes tiempos de encendido y apagado, secuencias específicas como el patrón SOS y el uso de millis() para medir el tiempo transcurrido. En conjunto, la práctica sirvió como una base sólida para entender la lógica de programación, la temporización y la observación de resultados en un sistema embebido.

VERDAD, BELLEZA, PROBIDAD

VERDAD, BELLEZA, PROBIDAD